Ders 9 Genişletilmiş Notu: Web Tarayıcıları: Mimari, Gerçekleştirme Sorunları ve Tercihli Tarama

Ders 9 Genişletilmiş Notu: Web Tarayıcıları: Mimari, Gerçekleştirme Sorunları ve Tercihli Tarama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu ve Planı
- 2. Tarayıcı Nedir, Nasıl Çalışır?
- 3. Tarayıcılara Neden İhtiyaç Duyarız?
- 4. Tarayıcıların Sınıflandırılması
- 5. Temel Sıralı Tarayıcı
- 6. Gezinme Stratejileri: Genişlik Öncelikli ve Derinlik Öncelikli
- 7. Perl ile Basit Bir Tarayıcı
- 8. Gerçekleştirme Sorunları (1): Frontier, Fetcher, Parser, Depo
- 9. Gerçekleştirme Sorunları (2): URL'ler, Örümcek Tuzakları ve Saklama
- 10. Eşzamanlı (Concurrent) Tarayıcı
- 11. Evrensel Tarayıcılar
- 12. Tercihli Tarayıcılar
- 13. Tercihli Tarama Algoritmaları
- 14. Tercihli Tarayıcıların Değerlendirilmesi
- 15. Açık Kaynak Tarayıcı Kütüphaneleri
- 16. Tarayıcı Uygulamalarında Karşılaşılan Pratik Sorunlar (Soru-Cevap)
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Konusu ve Planı

Hoca derse ders kitabının web tarama (web crawling) bölümüyle, yani 8. bölümle başlayacağını, vakit kalırsa 7. bölüm olan sosyal ağ analizine (social network analysis) geçeceğini söyledi. Derste vakit yalnızca tarayıcı bölümüne yetti.

Hocaya göre bu bölüm özellikle önemlidir, çünkü dönem projelerinin çoğu bir web tarayıcısı (web crawler) geliştirmeyi içeriyor. Bölümde bir tarayıcının ayrıntılı olarak nasıl gerçekleştirileceği (implementation) anlatılır. Dersin akışı şöyle planlandı:

- Tarayıcıların motivasyonu ve sınıflandırması (taxonomy): Ne tür tarayıcılar var, neden tarayıcılara ihtiyaç duyuyoruz?
- Temel bir tarayıcının mantıksal akışı ve modülleri.
- Gerçekleştirme sorunları (implementation issues).
- Evrensel tarayıcılar (universal crawlers): genel arama motorlarını destekleyen tarayıcılar ve onlara özgü sorunlar.
- Tercihli tarayıcılar (preferential crawlers) ve bunların iki türü: odaklı (focused) ve konulu (topical) tarayıcılar.

2. Tarayıcı Nedir, Nasıl Çalışır?

Hoca slaytta bir web araması resmiyle başladı. Bir anahtar kelime girersiniz, arama motoru size bir sonuç kümesi verir. Bu aramanın gerçekleşebilmesi için arama motoru bir tarayıcı uygulaması kullanır.

Tarayıcı uygulaması arka planda çalışır ve **aramadan önce** çalışır. Her gün çalışır, web'i tarar ve farklı HTTP sunucularında duran HTML sayfalarını indirir. Bütün HTML içeriğini merkezi bir yere indirir. Arama motoru da bu sayfaları indeksleyip aramayı bunun üzerinde yapar. Slayttaki örnekte, aranan anahtar kelimeye karşılık gelen belirli bir sayfanın bulunabilmesi için tarayıcının önce çok sayıda sayfayı indirmiş olması gerekiyordu.

2.1 Tohum URL'lerden Başlayan Döngü

Tarayıcı uygulamasına girdi olarak bir URL kümesi verilir. Bunlara tohum URL'ler (seed URLs) denir. Slayttaki örnekte tohum URL'ler Yahoo sayfasındandı. Tarayıcı şunları yapar:

1. Her URL için bir HTTP isteği (HTTP request) oluşturur.
2. Bu istekleri, URL'lere karşılık gelen HTML dokümanlarının bulunduğu HTTP sunucularına gönderir.
3. İsteğe yanıt olarak HTML dokümanlarını alır.
4. İndirdiği HTML sayfalarının içeriğinden yeni URL'leri çıkarır. Örnekte indirilen sayfalardan biri Yahoo'nun eğlence (entertainment) sayfasıydı ve ondan yeni URL'ler çıkarıldı.
5. Yeni URL'ler için yine HTTP isteği oluşturup bu yeni sayfaları da indirir.

Tarayıcı bunu bir **durma koşulu** (stopping condition) sağlanana kadar tekrarlar.

2.2 Durma Koşulu

Hoca burada "Tarayıcı web'i taramayı hangi koşulda durdurur?" diye sordu ve farklı durma koşulları olabileceğini söyledi:

- Tarayıcı belirli sayıda sayfa indirdiğinde durabilir.
- Tohum URL'lerden başlayarak tarama derinliği belirli bir seviyeye ulaştığında durabilir. Tohum URL'ler kök kabul edilirse birinci seviye, ikinci seviye, üçüncü seviye diye ilerlenir. Örneğin "üçüncü seviyeden sonra tarayıcım duracak" denebilir.

Yani her tarayıcının durma koşulu farklı olabilir.

2.3 Tarayıcının Diğer Adları ve Bant Genişliği

Tarayıcılar her gün düzenli olarak çalışır. Yalnız büyük arama motorlarının değil, başka amaçlarla düzenli çalışan pek çok tarayıcı da vardır. Bu uygulamalar farklı adlarla anılabilir ama hepsi aynı şeyi ifade eder: crawler, spider, robot, web agent. Bilinen örnekler Google'ın tarayıcısı Googlebot ve hocanın "MSNbot" diye andığı Microsoft tarayıcısıdır.

Hoca, tahmini olarak internet bant genişliğinin %60'ının tarayıcılar tarafından tüketildiğini söyledi. Bunun sebebi tarayıcıların farklı HTTP sunucularından sürekli sayfa indirmesidir. Bunu sürekli yaparlar, çünkü indekslerini hem olabildiğince **büyük** hem de olabildiğince **taze** (fresh), yani güncel tutmak isterler.

> **Ders sonrası not:** Microsoft'un tarayıcısı için hoca "MSNbot" dedi. MSNbot büyük olasılıkla 2010 civarında Bing'in tarayıcısı Bingbot ile değiştirildi. "Bant genişliğinin %60'ı tarayıcılara gidiyor" ifadesi hocanın da belirttiği gibi bir tahmindir; yayımlanan bot trafiği ölçümleri büyük olasılıkla web trafiğinin yaklaşık %40'ını iyi ve kötü tüm botlara bağlar ve bunun yalnız bir kısmı arama motoru tarayıcılarıdır.

3. Tarayıcılara Neden İhtiyaç Duyarız?

Hoca tarayıcıların kullanım amaçlarını şöyle sıraladı:

- **Arama motorlarını desteklemek:** Hepimiz web'i aramak için arama uygulamaları kullanıyoruz. Önceki bölümde (bilgi erişimi ve web arama motorları) gördüğümüz gibi, gerçek zamanlı aramanın tamamı indeks üzerinde yapılır. Aramanın yapılabilmesi için önce bir

indeks, yani ters indeks (inverted index) oluşturulabilmelidir. Tarayıcılar HTML sayfalarını merkezi bir yere indirerek bu indeksin oluşturulmasını sağlar.
- **Dikey (vertical) ya da özelleşmiş (specialized) tarayıcılar:** Belirli türdeki siteler için çalışan tarayıcılardır. Haber siteleri, alışveriş siteleri, makaleler (papers), yemek tarifleri, yorumlar (reviews) için özel tarayıcılar vardır. Dönem projelerinin bir kısmı da böyle bir dikey tarayıcıdır: Başka sitelerden bilgi çıkarıp kullanıcıya birleşik (hibrit) bir biçimde sunan özelleşmiş bir site.
- **İş zekâsı (business intelligence):** Bir işletme rakiplerinin ne yaptığını öğrenmek isterse, rakiplerin sitelerini tarayan bir tarayıcı kullanır.
- **Siteleri izlemek (monitoring):** İlgilendiğiniz sitelerde yeni etkinlik ya da güncelleme olup olmadığını tarayıcıyla takip edebilirsiniz.
- **Kötü amaçlı kullanım:** Bazı tarayıcılar özellikle istenmeyen e-posta (spam) için kullanılır. Bu tarayıcılar web'den e-posta adreslerini toplar (harvesting) ve bir e-posta veritabanı oluşturur. Bu veritabanı daha sonra spam ya da oltalama (phishing) gibi amaçlarla kullanılabilir.

3.1 Web Madenciliği Resmindeki Yeri

Hoca daha önce gösterdiği bir resmi tekrar gösterdi. O resimde web içerik madenciliğine (web content mining) odaklanmıştık: Bütün siteler merkezi bir yere indirildikten sonra metin analizi yapılıyor, ters indeks oluşturuluyor ve arama bu ters indeks üzerinde yapılıp aramaya karşılık gelen sonuçlar (hits) üretiliyordu. Aynı resimde bir uygulama daha vardı: tarayıcı. Tarayıcı, web'deki HTML dokümanlarını bulmaktan, indirmekten ve merkezi bir sayfa deposunda (page repository) saklamaktan sorumludur. Bu derste o uygulama anlatıldı.

4. Tarayıcıların Sınıflandırılması

Hoca slayttaki sınıflandırma şemasını adım adım açıkladı. Tarayıcılar iki ana kategoriye ayrılır:

- **Evrensel tarayıcılar (universal crawlers):** Genel arama motorlarını destekler. Her türlü HTML dokümanını bulmaktan sorumludur.
- **Tercihli tarayıcılar (preferential crawlers):** Yalnızca belirli bir konudaki (topic) HTML dokümanlarını bulur.

Tercihli tarayıcılar da iki kategoriye ayrılır.

4.1 Odaklı Tarayıcı (Focused Crawler)

Odaklı tarayıcı, yine belirli bir konu için **indeks oluşturmaktan** sorumludur. Hocanın örneği: elektronik e-ticaret siteleri için bir odaklı tarayıcı. Bu, elektronik e-ticaret sitelerinin bir indeksini oluşturmak istediğiniz anlamına gelir. Böylece odaklı bir arama motorunu desteklersiniz: Arama motorunuz yalnızca elektronik e-ticaret siteleri üzerinde arama yapmaya izin verir.

4.2 Konulu Tarayıcı (Topical Crawler)

Konulu tarayıcıda **indeks oluşturulmaz**. Bunun yerine tarama, arama sırasında yapılır. Kullanıcı bir arama sorgusu girip arama düğmesine bastığı anda konulu tarayıcı bu konuya uyan HTML sayfalarını aramaya başlar. Konuya uyan sayfaları buldukça indirir ve sonuçları grafik kullanıcı arayüzünde son kullanıcıya gösterir.

Konulu tarayıcı da web'i nasıl taradığına göre ikiye ayrılır:

- **Statik konulu tarayıcı (static topical crawler):** Web'i önceden tanımlanmış (predefined) bir tarama algoritmasına göre gezer. Bu algoritma PageRank tabanlı gezinme, en iyi öncelikli (best-first), genişlik öncelikli (breadth-first) ya da derinlik öncelikli (depth-first) gezinme olabilir. Tarayıcının web'i nasıl gezeceğini bu algoritmalar zaten tanımlar.
- **Uyarlamalı konulu tarayıcı (adaptive topical crawler):** Taradığı sayfalardan gerçek zamanlı olarak öğrenir.

Hocanın uyarlamalı tarayıcı örneği şuydu: "web data mining" anahtar kelimesiyle aramaya başladık. Web'i tararken yeni HTML sayfalarıyla karşılaşırız ve bu sayfalardan yeni anahtar kelimeler çıkarırız. Diyelim ki bir sayfada "information retrieval" anahtar kelimeleri de yüksek

sıklıkta geçiyor. Bu durumda uyarlamalı konulu tarayıcı “information retrieval” kelimelerini “web data mining” kelimelerine ekler. O andan itibaren yalnızca “web data mining” değil, “web data mining” ve “information retrieval” birlikte aranır. Tarayıcı, taradığı sayfalardan gerçek zamanlı öğrendiği için uyarlamalıdır. Bu tür tarayıcıların farklı türleri vardır, ama hepsi aynı şeyi yapar: Taranan sayfalardan gerçek zamanlı öğrenirler.

5. Temel Sıralı Tarayıcı

Hoca ardından temel sıralı tarayıcının (basic sequential crawler) akış şemasını anlattı. Şemadaki dikdörtgenler modülleri, farklı biçimdeki şekil ise girdiyi gösterir. Girdi, tohum URL’lerdir. Her tarayıcı uygulaması başlangıçta kullanıcıdan tohum URL’leri alır.

Modüller sırasıyla şöyledir:

1. **Frontier’ı başlatma (initialize frontier):** Tohum URL’leri alıp bir frontier listesi oluşturur. Frontier listesi, sırada taranmayı bekleyen URL’lerin listesidir. Genellikle bağlı liste (linked list) ile gerçekleştirilir ve fiziksel bellekte tutulur.
2. **Kuyruktan alma (dequeue):** Frontier listesinin başındaki (head) düğümü alır, bu düğümden URL’yi çıkarır ve bir sonraki modüle verir.
3. **İndirme (fetch):** URL’yi alır, bir HTTP isteği oluşturur ve bunu internet üzerinden URL’nin gösterdiği HTTP sunucusuna gönderir. Yanıt olarak bir HTTP yanıt nesnesi (HTTP response object) gelir. Fetch modülü bu nesneden HTML sayfasını çıkarıp bir sonraki modüle verir.
4. **Ayrıştırma (parse):** Kendisine verilen her HTML dokümanından URL’leri çıkarır ve bu URL’leri frontier listesine geri koyar. Ardından HTML dokümanını bir sonraki modüle verir.
5. **Saklama (store):** HTML dokümanını depoya (repository) yazar.
6. **Durma koşulu kontrolü:** Durma ölçütüne ulaşılmadıysa yeniden dequeue modülüne dönülür ve frontier listesinden başka bir URL alınır.

Hoca burada tarayıcının iki görevi olduğunu vurguladı. Birincisi, URL’leri çıkararak sırada hangi URL’lere gidileceğini bulmaktır. İkincisi, indeksin oluşturulabilmesi için HTML doküman deposunu oluşturmaktır. İndeks, tarayıcının taradığı bütün HTML dokümanları kullanılarak oluşturulur.

Durma ölçütü farklı olabilir: Frontier listesinde hiç URL kalmayana kadar devam edilebilir ya da indirilmesi gereken en fazla URL sayısına ulaşılana kadar devam edilebilir.

6. Gezinme Stratejileri: Genişlik Öncelikli ve Derinlik Öncelikli

Web grafiğini gezmenin farklı yolları vardır. Bu da tarayıcıyı gerçekleştirmenin farklı yolları olduğu anlamına gelir. Hoca yaygın kullanılan iki statik stratejiyi gösterdi.

6.1 Genişlik Öncelikli Arama (Breadth-First Search)

Genişlik öncelikli arama web’i seviye seviye (level-wise) tarar. İlk giren ilk çıkar (first-in-first-out, FIFO) türünde bir gezinmedir: Frontier listesine ilk konan düğüm, listeden yine ilk alınır.

Slayttaki örnekte tohum URL, taranan web ağacının kök düğümüdür. Kök indirilir ve URL’leri çıkarılır; iki URL çıkar. Genişlik öncelikli arama önce birinci düğümü, sonra ikinci düğümü indirir. Sonra bu iki düğümden URL’leri çıkarır. Ardından birinci düğümün çocuklarını indirir, bitirince ikinci düğüme dönüp onun çocuklarını indirir. Böylece web seviye seviye taranır.

6.2 Derinlik Öncelikli Arama (Depth-First Search)

Derinlik öncelikli aramada da önce kök düğüm indirilir; iki çocuğu vardır. Bu kez birinci çocuktan başlanır ve indirilir. Onun da iki çocuğu vardır; ilk çocuğu indirilir. Onun da çocukları varsa ilk çocuğa inilir. Çocuğu olmayan bir düğüme gelince bir seviye geri çıkılır ve ikinci çocuk indirilir. Bir düğümün bütün çocukları bitince bir seviye daha geri çıkılıp üst seviyedeki bir sonraki düğüme geçilir. Böylece bir daldaki (branch) bütün düğümler bitirilir, sonra ikinci dala geçilir ve o daldaki bütün düğümler indirilir.

Bu gezinme son giren ilk çıkar (last-in-first-out, LIFO) türündedir. Hoca veri yapıları dersini hatırlattı:

- FIFO gerçekleştirmesinde **kuyruk** (queue) kullanılır: genişlik öncelikli.
- LIFO gerçekleştirmesinde **yığın** (stack) kullanılır: derinlik öncelikli.

7. Perl ile Basit Bir Tarayıcı

Hoca slaytta Perl dilinde yazılmış basit bir sıralı tarayıcı gösterdi ve kodun okunmasının kolay olduğunu söyledi. Kodun yapısı, kısa nottaki ve hocanın satır satır anlatımındaki haliyle şöyledir:

```
my @frontier = read_seeds($file);
while (@frontier && $tot < $max) {
    my $next_link = shift @frontier;
    my $page = fetch($next_link);
    add_to_index($page);
    my @links = extract_links($page, $next_link);
    push @frontier, process(@links);
}
```

Hocanın satır satır açıklaması:

- Bir dosyada bir URL kümesi vardır. Dosyadaki bütün URL'ler okunup frontier listesine konur. Bu liste, sırada hangi URL'lerin taranacağını bulmak için kullanılmaya devam edilecek listedir.
- Frontier listesi oluşturulduktan sonra bir `while` döngüsü başlar. Bu örnekte durma koşulu indirilmesi gereken toplam URL sayısıdır: İndirilen toplam sayı (`$tot`) en fazla sayıdan (`$max`) küçük olmalıdır. Bu üst sınır yapılandırılabilir (configurable) bir parametredir. Tarayıcıya tohum URL'lerin yanında bu değer de verilmelidir. Döngü, frontier listesinde URL kalmayana kadar ya da en fazla URL sayısına ulaşılana kadar sürer.
- Döngünün başında frontier listesindeki ilk URL alınır ve geçici bir değişkene (`$next_link`) atanır. Taranacak URL budur.
- `fetch` fonksiyonu bu URL'ye karşılık gelen HTML sayfasını indirir; sayfa `$page` değişkenine atanır.
- Sayfa depoya eklenir (`add_to_index`).
- Sayfadan bağlantılar çıkarılır (`extract_links`). Bağlantıları çıkarırken, çıkarılan URL'ler arasında mevcut sayfanın kendi URL'sinin (`$next_link`) olmadığından emin olunur. Amaç aynı sayfayı tekrar tekrar indirmemektir.
- Çıkarılan URL'ler `@links` değişkenine atanır ve frontier listesinin sonuna geri itilir (`push`).
- Sonra döngü başa döner.

Listenin başından `shift` ile alıp sonuna `push` ile eklemek, frontier'ı bir kuyruk gibi kullanmak demektir. Yani bu kod genişlik öncelikli bir tarayıcıdır (ara adım eklendi).

Hoca, kodun basit görünmesine rağmen tarayıcının her adımında çözülmesi gereken gerçekleştirme sorunları olduğunu söyledi ve bunlara geçti.

8. Gerçekleştirme Sorunları (1): Frontier, Fetcher, Parser, Depo

8.1 Aynı Sayfayı Tekrar İndirmemek

Aynı sayfayı iki kez indirmek istemeyiz. Bunun için ziyaret edilmiş sayfaları gösteren bir tablo tutulur. Perl kodunda sayfadan URL'leri çıkarırken mevcut sayfanın URL'sinin aralarında olmamasına dikkat etmiştik. Mevcut sayfanın URL'si frontier'a konursa tarayıcı aynı dokümanı yeniden indirir. Genel kural şudur: Herhangi bir URL frontier listesine geri konmadan önce o URL'nin daha önce indirilip indirilmediği kontrol edilmelidir.

8.2 Frontier Listesinin Hızlı Büyümesi

İkinci sorun, frontier listesinin çok hızlı büyümesidir. Her sayfa indirildiğinde ondan çok sayıda URL çıkar ve frontier'a konur. Bir URL birçok yeni URL üretir; bu böyle devam ederse frontier çok hızlı büyür.

Elbette verilen bir sürede olabildiğince çok sayfa taramak isteriz, ama aynı zamanda **önemli** sayfaları taramak isteriz. Bu yüzden yeni çıkarılan URL'leri frontier'a koyarken onları önemlerine göre önceliklendiren bir sırayla koyarız. Önemi yüksek URL'ler frontier listesinin başına konur. Böylece tarama için ayrılan sürede önemsiz sayfalar yerine önemli sayfalar taranır.

8.3 Fetcher: Zaman Aşımı

Fetcher modülü bir HTTP isteği oluşturur, HTTP sunucusuna gönderir ve bir HTTP yanıt nesnesi alır. Bu süreç ağ gecikmesini (network latency) içerir. HTTP sunucusu web'de ulaşılması uzun süren bir yerde olabilir, ya da o sunucuya giden bant genişliği sınırlı olabilir. Bu yüzden fetcher bazı sayfalar için uzun süre bekleyebilir.

Bunu istemeyiz; fetcher'ın sınırlı süre beklemesini isteriz. Bu yüzden fetcher gerçekleştirilirken bir zaman aşımı (timeout) konur. HTTP isteği gönderildikten sonra belirli bir zaman aşımı süresi içinde yanıt gelmezse fetcher o sayfayı almaktan vazgeçer ve başka bir URL'ye geçer.

> **Hocanın vurgusu:** Fetcher modülü sağlam (robust) olmalı ve bir zaman aşımı mekanizması içermelidir.

8.4 Fetcher: Yalnız İşlenebilir Doküman Türleri

Web'den her türlü dokümanı indirmek istemeyiz; yalnızca ayrıştırıp (parse) indeksleyebileceğimiz dokümanları indiririz. Web'de farklı uzantılara sahip farklı türde dokümanlar vardır. Yalnızca HTML, Word ve metin dokümanları gibi ayrıştırıp indeksleyebileceğimiz türler indirilir.

Bir dokümanın istenen türde olup olmadığını anlamak için önce dokümanın içerik türünü (content type) getiren bir HTTP komutu gönderilir: `HEAD`. Gelen içerik türü ayrıştırabileceğimiz bir türse, dokümanı almak için ikinci bir HTTP isteği gönderilir. Dokümanı almak için kullanılan komut genellikle `GET`'tir.

> **Ders sonrası not:** Altyazıda "hat" diye geçen komut HTTP `HEAD` yöntemidir.
>
> **Ek bilgi (derste anlatılmadı):** HTTP'de `HEAD` isteği, `GET` ile aynı başlıkları (headers) döndürür ama yanıtın gövdesini (body) göndermez. Bu yüzden `Content-Type` ve `Content-Length` gibi başlık alanlarını sayfanın tamamını indirmeden öğrenmenin ucuz bir yoludur. Bazı sunucular `HEAD` isteğini desteklemeyebilir ya da başlıkları eksik gönderebilir; bu durumda tarayıcılar `GET` yanıtının başlıklarına bakıp indirmeyi yarıda kesebilir.

8.5 Fetcher: Boyut Sınırı

Zaman aşımına ek olarak bir kural daha konabilir: Yalnızca en fazla 100 kilobayt boyutundaki dokümanlar indirilir. Bazen indirebileceğimiz türde (HTML ya da Word) bir doküman çok büyük olabilir. Bu durumda iki seçenek vardır:

- Dokümanın boyutu 100 KB'ın üzerindeyse hiç indirilmez.
- Her dokümanın yalnızca ilk 100 KB'ı indirilir, yani dokümanın tamamı yerine bir kısmı alınır.

8.6 Ayrıştırma (Parsing) ve DOM Ağacı

Sayfa indirildikten sonra URL'leri çıkarmak için HTML dokümanının ayrıştırılması gerekir. Fakat web'deki HTML dokümanları çoğu zaman iyi yazılmamıştır. HTML dokümanlarının belirli dilbilgisi kurallarına uyması ve ayrıştırıcıların içeriği doğru anlayabilmesi için belirli etiketleri (tags) içermesi gerekir. Çoğu zaman sayfa yazarları bir editör kullanmadan sayfalarını elle yazdıkları için ya da başka sebeplerle, ayrıştırıcılar için önemli olan bazı etiketler eksik kalır.

Hoca bunun çok önemli olduğunu söyledi. Bugün kullandığımız Google Chrome, Microsoft Edge gibi tarayıcılar, eksik öğeler olsa bile HTML dokümanlarını ayrıştırabilir. Eksik kısımları akıllıca anlayıp HTML'i kurar, işler (render) ve kullanıcıya gösterirler. Tarayıcımızdaki ayrıştırıcı da aynı şekilde çalışmalı, eksik öğeleri olan HTML dokümanlarını da anlayabilmelidir.

Bunun için ayrıştırıcı gerçekleştirmelerinde belge nesne modeli (Document Object Model, DOM) kullanılır ve HTML dokümanından DOM ağacı (DOM tree) denen bir ağaç kurulur. DOM ağacı kurulduktan sonra içerik ayrıştırılır ve dokümana gömülü URL'ler çıkarılır.

> **Ek bilgi (derste anlatılmadı):** DOM ağacında HTML dokümanındaki her etiket bir düğümdür; örneğin `<html>` kök, `<body>` onun çocuğu, sayfadaki her `<a>` bağlantı etiketi de bir yaprağa yakın düğümdür. URL çıkarmak, bu ağaçta `<a>` düğümlerini gezip `href` özniteliklerini (attribute) okumak demektir. Hoşgörülü (lenient) ayrıştırıcılar kapanmamış etiketleri kurallara göre kendiliğinden kapatarak bozuk HTML'den de bir ağaç kurabilir.

8.7 Depo ve Metin İşleme

Hoca bu noktaya kadar frontier'dan URL alma ve frontier'a URL koyma sorunlarını, fetcher'ın sağlam olması gerektiğini ve URL çıkarmak için DOM ağacıyla gelişmiş bir ayrıştırıcı kurulabileceğini konuştuklarını özetledi. Ardından HTML sayfalarından oluşturulan depoya geçti.

Depoya yazmanın iki yolu vardır:

- HTML dokümanları depoya **olduğu gibi** (as is) yazılır ve indeks oluşturmak için depo sonradan işlenir.
- Sayfa depoya yazılırken içeriği çıkarılır ve depoya içerik yazılır.

İçerik çıkarılırken anahtar kelimelerin nasıl çıkarıldığına dikkat etmek gerekir, çünkü dokümandaki her kelimeyi almak istemeyiz:

- **Durak kelimeleri çıkarma (stop word removal):** Sayfanın konusuyla ilgili olmayan durak kelimeler atılır.
- **Eş anlamlıları birleştirme (conflation, synonym processing):** Arama motorunun ve web içerik madenciliği modülünün daha küçük bir indeks oluşturabilmesi için benzer anlamlı kelimeler birleştirilebilir. Dokümanda benzer anlamlı birkaç kelime geçiyorsa, hepsini koymak yerine hepsini temsil eden tek bir kelime konur.
- **Kök bulma (stemming):** Aynı anlama gelen kelimelerin her biçimini saklamak yerine kelimenin kökü bulunur ve indekse o kök yazılır.

Hoca, tarayıcıların çoğunun HTML dokümanlarını depoya hiçbir değişiklik yapmadan olduğu gibi yazdığını vurguladı. Durak kelime çıkarma, kök bulma ve eş anlamlı birleştirme bu üç işlem genellikle depo oluşturulduktan sonra, indeks oluşturulurken depo üzerinde uygulanır.

Hoca bu noktada beş dakikalık bir ara verdi.

9. Gerçekleştirme Sorunları (2): URL'ler, Örümcek Tuzakları ve Saklama

9.1 Göreli ve Mutlak URL

Bir HTML sayfasından URL'ler çıkarıldığında bazılarının mutlak (absolute), bazılarının göreli (relative) URL olduğu görülür. Hoca "Fark nedir?" diye sordu ve slayttaki örnekle açıkladı. Slayttaki `intl.html` dokümanına giden bağlantı mutlak bir URL'dir: `http` ile başlar ve tıklandığında tarayıcı sizi doğrudan o siteye götürür.

Fakat HTML dokümanının editörü, bağlantı aynı alan adı (domain) içindeyse mutlak URL yerine göreli bir URL koyabilir. Bu yüzden HTML dokümanı ayrıştırılırken karşılaşılan URL'nin göreli mi mutlak mı olduğuna dikkat edilmelidir. Göreli bir URL ise, üst sayfanın sitesi olan temel URL (base URL) alınır ve göreli URL bunun sonuna eklenir. Böylece mutlak bir URL elde edilir.

> **Hocanın vurgusu:** Frontier listesine konacak bütün URL'ler mutlak URL olmalıdır.

9.2 Kanonik URL

Bazen aynı URL'nin farklı yazımlarıyla karşılaşılır. Slayttaki dört URL'nin hepsi aynı sayfayı gösteriyordu, ama sayfanın yazarı farklı sebeplerle bunları farklı biçimlerde yazmıştı:

- Birinde küçük harf, birinde büyük harf kullanılmış.

- Birinde, gerekli olmadığı halde port numarası yazılmış.
- Birinde önce bir dizin seviyesi içeri girilip sonra üst seviyeye geri çıkılarak (`../` ile) aynı dizine ulaşılmış.

Hepsi aynı sayfadır. Bu yüzden HTML sayfasından URL'ler ayrıştırılırken bir **kanonik biçim** (canonical form) belirlenir. Kanonik biçim, bunların hepsine karşılık gelen tek bir URL çıkarmamızı sağlar. Slaytta kanonik biçime uymayan URL'ler ve onların kanonik biçime uyan halleri örnek olarak gösterildi.

> **Hocanın vurgusu:** Frontier listesine konacak bütün URL'ler kanonik biçimde olmalıdır.

9.3 Örümcek Tuzakları (Spider Traps)

Bir URL frontier'a konurken bir örümcek tuzağının parçası olmadığından emin olunmalıdır. Örümcek tuzakları genellikle CGI betikleri (CGI scripts) ya da başka tür betikler içerir. Her tıklamada bu betikler HTML sayfası içinde farklı URL'ler üretir. Yeni üretilen URL'ler de CGI betiği içerir; onlara tıklanınca yine başka CGI URL'leri üretilir. Amaç, tarayıcıyı aynı tuzağın, yani aynı alt alan adının (subdomain) içinde taramaya devam ettirmektir.

Bunu önlemek için bir URL çıkarıldığında karakter sayısı sayılabilir ve bir eşik (threshold) konabilir. Slayttaki örnekte eşik 128 karakterdir. URL'de 128'den fazla karakter varsa bu URL göz ardı edilir ve frontier'a konmaz.

> **Ders sonrası not:** 128 karakter, slayttaki örnek eşiktir; standart bir değer değildir. Kanonik URL ve örümcek tuzağı kuralları sezgisel (heuristic) kurallardır.

9.4 Sayfaların Saklanması

Saklama (storage) modülüne dair gerçekleştirme sorunları şunlardır:

- **Ayrı dosyalar:** HTML sayfaları dosya sisteminde ayrı dosyalar olarak saklanabilir. Dosyalara isim verirken URL, bir özet fonksiyonu (hashing function) ile dosya adına eşlenebilir. Böylece her HTML dokümanı URL'sinin özetinden üretilen bir ad alır. Fakat bu çok sayıda dosya üretir: Küçük bir tarayıcıyla bile milyonlarca sayfa elde edilir ve dosya sisteminde duran milyonlarca dosyayı yönetmek zordur.
- **XML:** HTML dokümanları büyük tek bir XML dokümanının iç etiketlerine yerleştirilir. Böylece saklanmak istenen bütün HTML dokümanlarının içeriğini tutan büyük tek bir XML dokümanı elde edilir.
- **Veritabanı:** SQL Server gibi ilişkisel veritabanı yönetim sistemlerinin yükü (overhead) fazladır. Bu yüzden HTML sayfalarını saklamak için Berkeley DB gibi hafif (lightweight) bir veritabanı kullanılması önerilir.

10. Eşzamanlı (Concurrent) Tarayıcı

Şimdiye kadar gösterilen sıralı tarayıcı tek bir iş parçacığında (single thread) çalışır. Web'den sayfaları kısa sürede taramak istediğimiz için tarayıcının eşzamanlı (concurrent) ya da paralel bir sürümünü gerçekleştirmemiz gerekir. Bunun için önce sıralı tarayıcının hangi kısımlarının çok zaman aldığına bakmak gerekir.

10.1 Sıralı Tarayıcıda Zaman Alan İşler

DNS çözümleme: Fetcher modülü bir URL'ye karşılık gelen HTML dokümanını alırken önce URL'yi bir DNS sunucusuna gönderir. DNS sunucusu IP adresini geri gönderir ve fetcher HTTP isteğini bu IP adresiyle oluşturur. DNS sunucusuna IP adresini sormak zaman alır ve bu bütün URL'ler için yapılmak zorundadır. Bu yüzden DNS ad çözümlemenin (DNS name resolving) hızlandırılıp hızlandırılamayacağına bakılmalıdır.

Senkron soketler: Ağ iletişiminde TCP/IP bağlantısı kurmaya izin veren bir kütüphane kullanılır. C, C#, Java ve Python kütüphanelerinin hepsi HTTP isteği oluşturup göndermek için TCP/IP protokolünü kullanır. Bu kütüphanelerde soket (socket) adında bir sınıf vardır. Bir soket nesnesi oluşturulurken iletişimin yapılacağı hedef makinenin IP adresi ve port numarası tanımlanmalıdır. Soket sınıflarının farklı türleri vardır:

- **Senkron soket (synchronous socket):** Soketi kullanan istemci uygulaması, yanıt nesnesi gelene kadar bekler.
- **Asenkron soket (asynchronous socket):** HTTP isteği gönderilir ve yanıt gelene kadar beklemek gerekmez. Bu sırada soket başka bir iş parçacığı tarafından başka bir HTML sayfasını almak için kullanılabilir.

Senkron soket kullanmak da çözülmesi gereken bir sorundur.

Sayfanın alınması: İstenen sayfanın yanıt olarak alınması da zaman alır. Bu, soket sorunuyla bağlantılıdır. Soket bir şekilde asenkron yapılabilirse istenen sayfa için beklemek gerekmez.

10.2 Çok İş Parçacıklı Mimari

Hoca slaytta sıralı tarayıcının çok iş parçacıklı (multi-threaded) sürümünü gösterdi:

1. Girdi yine tohum URL'lerdir.
2. Sıralı tarayıcıdaki "frontier'ı başlatma" modülünün yerini bir **frontier yöneticisi** (frontier manager) alır. Frontier yöneticisi tohum URL'leri alıp frontier listesine koyar. Frontier listesi aynı kalır: Bir veri yapısıdır ve genellikle bağlı listeyle gerçekleştirilir.
3. Frontier yöneticisi bütün tarayıcı iş parçacıklarını başlatır. Şemada gri arka planlı her dikdörtgen bir tarayıcıdır ve bir iş parçacığı olarak gerçekleştirilir.
4. Bir iş parçacığı başlatıldığında frontier yöneticisi üzerinden frontier listesine gider ve indirmek için bir URL ister. Frontier yöneticisi URL'yi alıp ona verir.
5. URL fetcher'a verilir. Fetcher sayfayı web'den alıp HTML dokümanını ayrıştırıcıya verir.
6. Ayrıştırıcı URL'leri çıkarır ve yine frontier yöneticisine gidip bu URL'lerin frontier listesine konmasını ister.
7. Ayrıştırıcı işini bitirince HTML dokümanını saklama modülüne verir; saklama modülü dokümanı depoya yazar.
8. İş parçacığı işinin bitip bitmediğini kontrol eder. Bitmediyse aynı şeyi yapmaya devam eder.

Yüzlerce farklı iş parçacığı olduğunu düşünün: Hepsi frontier listesinde hiç URL kalmayana kadar aynı işi aynı anda yapar.

Eşzamanlı tarayıcı çok süreçli (multi-processing) ya da çok iş parçacıklı (multi-threading) olarak gerçekleştirilebilir; ikisi de olur. Her süreç ya da iş parçacığı bir sıralı tarayıcı gibi çalışır ve sıralı tarayıcının adımlarını içerir.

10.3 Paylaşılan Veri Yapıları ve Senkronizasyon

Tek fark, bütün iş parçacıklarının paylaşılan veri yapılarını (shared data structures) ortak kullanmasıdır: **frontier** ve **depo**.

- **Depo:** Aynı sayfanın aynı anda iki kez indirilmediğini kontrol etmenin bir yolu varsa, depoya farklı HTML dokümanları yazmak sorun olmaz. Her yazmada farklı bir HTML sayfası yazılacağı için yazma çakışması (write conflict) olmaz.
- **Frontier:** Frontier listesinden URL almak için birden fazla iş parçacığı listeye erişiyor olabilir. Bu yüzden burada senkronizasyon gerekir.

  Hocanın vurgusu: Frontier paylaşılan bir veri yapısıdır. Senkronizasyon frontier yöneticisinde yapılır.

Hoca, bugün bütün tarayıcıların çok iş parçacıklı, yani eşzamanlı olarak gerçekleştirildiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Frontier yöneticisindeki senkronizasyon genellikle bir kilit (lock, mutex) ya da iş parçacığı güvenli (thread-safe) bir kuyrukla yapılır. Kilit yoksa iki iş parçacığı listenin başını aynı anda okuyup aynı URL'yi alabilir; bu bir yarış durumudur (race condition). Ziyaret edilmiş URL tablosuna "kontrol et ve ekle" işleminin de tek bir bölünmez (atomic) adımda yapılması gerekir, yoksa aynı sayfa yine iki kez indirilebilir.

11. Evrensel Tarayıcılar

Evrensel tarayıcılar, evrensel arama motorlarını destekleyen tarayıcılardır. Google, Bing, Yahoo ve Yandex'in kendi evrensel tarayıcıları vardır. Bu tarayıcılar büyük ölçeklidir ve daha önce söylendiği gibi ağ bant genişliğinin büyük bir kısmını tüketir. HTML sayfalarını indirirken indeksi sıfırdan oluşturmazlar; indeks **artımlı** (incremental) olarak oluşturulur.

11.1 Kapsam ile Tazelik Dengesi

Bu tarayıcılarda performans ve politika (policy) ile ilgili bir denge (trade-off) vardır:

- **Kapsam (coverage):** Tarayıcıyı milyarlarca sayfaya ölçeklemek, yani indeksin boyutunu olabildiğince büyük tutmak istersiniz. Bir tarayıcının performansı, web'in ne kadar büyük bir kısmını kapsadığıyla ölçülür. Bu politikada tarayıcı, verilen sürede olabildiğince çok sayfa tarayacak şekilde gerçekleştirilir; HTTP istekleri sürekli farklı sayfalara yapılır.
- **Tazelik (freshness):** İndeksteki URL'lerin güncel olup olmadığıdır. Hocanın örneği: Bir HTML sayfasını iki ay önce indirip indekse koydunuz. Bu iki ay içinde sayfanın içeriği çok değişti, ama siz aynı sayfayı yeniden taramadınız. O URL indeksinizde yanlış içerikle duruyor, çünkü sayfanın güncel içeriği değişti. İndeks tablosunu güncel tutmak için daha fazla sayfa taramak yerine aynı sayfalar belirli bir sıklıkla yeniden taranabilir. Bu durumda indeks daha küçük olabilir, ama çaba indeksin içeriğini güncel tutmaya harcanır.

Yani ya milyarlarca sayfaya ölçeklenirsiniz ya da yalnızca güncel bir indeks tutarsınız. Büyük evrensel tarayıcılar gerçekleştirmelerini bu iki konu arasında yapılandırır.

Hoca slaytta bir tarayıcının yıllara göre indeks boyutunu gösteren bir örnek gösterdi. İndeks boyutları yıldan yıla değişebilir. Bunun sebebi yine bu dengedir: Bazı yıllarda kapsama, bazı yıllarda tazeliğe ağırlık verilebilir ve bu da evrensel tarayıcının indeks boyutunu etkiler.

11.2 Darboğazlara Çözümler

Bir tarayıcıda zaman alan kısımlar DNS sunucusu, saklama ve soket kullanımıdır. Büyük ölçekli tarayıcılar bunların her biri için çözüm geliştirir. Mimari, eşzamanlı tarayıcıyla aynıdır; fark bir iş parçacığının içindeki fetcher'dadır:

- **DNS önbelleği:** Fetcher'ın içinde DNS çözümleme (resolve DNS) alt bileşeni vardır ve URL'ye karşılık gelen IP adresini bulmaktan sorumludur. Evrensel tarayıcılar, IP adreslerini DNS'e sormak yerine önbellekten (cache) almalarını sağlayan, DNS sunucularıyla bütünleşik bir önbellek mekanizması kullanır.
- **Asenkron soketler:** Senkron soket yerine asenkron soket kullanılır. Böylece bir HTTP isteği yapıldığında iş parçacığı bloklanmaz.
- **Yazılım tabanlı ağ anahtarları (software-based network switches):** HTML dokümanlarını web'den alma gecikmesini iyileştirir.
- **Yazılım tabanlı depolama anahtarları (software-based storage switches):** HTML dokümanlarının depoya hızlı yazılmasını sağlar.

Yani evrensel tarayıcılar yalnızca sıralı tarayıcının eşzamanlı sürümünü gerçekleştirmekle kalmaz, daha fazla zaman isteyen işlem adımlarına da çözüm bulur.

11.3 Tazeliğin Belirlenmesi

Hoca "Bir web sayfasının tazeliğini nasıl tanımlarız?" diye sordu. Bunun için HTTP başlığındaki (HTTP header) `Last-Modified` ve `Expires` öğeleri kullanılır:

- `Last-Modified` alanı, HTML sayfasının en son ne zaman güncellendiğini anlamamızı sağlar. Buna göre indirilen sayfanın güncel olup olmadığını anlarız.
- `Expires` alanı bir tarih verir. Bu tarihten sonra sayfanın içeriği geçersiz olacak, yani sayfanın yazarı sayfayı güncelleyecek demektir. Bu yüzden sayfanın içeriği o tarihte gidip yeniden alınmalıdır. Böylece sayfanın hangi tarihte yeniden taranması gerektiği de anlaşılır.

  Ders sonrası not: Hoca bir yerde "HTML dokümanındaki `Expires` öğesi" dedi. `Last-Modified` ve `Expires`, hocanın da ilk söylediği gibi HTML dokümanının değil HTTP yanıt başlığının alanlarıdır.

Hoca bu konunun daha fazla ayrıntısını veren bazı slaytları atladı.

12. Tercihli Tarayıcılar

Tercihli tarayıcı, belirli bir konuda arama yapan arama motorlarını oluşturmaktan ya da desteklemekten sorumludur. Frontier listesi oluşturulurken, tarayıcının konusuyla aynı içeriğe sahip HTML dokümanları önceliklendirilir.

Her tercihli tarayıcının bir konusu vardır: yemek tarifi tarayıcısı, haber tarayıcısı, e-ticaret tarayıcısı gibi. Dönem projesinde tarayıcı geliştirecek öğrenciler aslında bir tercihli tarayıcı geliştiriyor. Örneğin bazı gruplar televizyonlar ya da iPhone için bir tercihli tarayıcı yapmak istemişti. Bu durumda birden fazla e-ticaret sitesine gidilir, bu sitelerden sayfalar alınır ve sayfalar frontier listesine konurken içeriklerinin tarayıcının konusuna ne kadar benzediğine göre sıralanır.

12.1 Odaklı Tarayıcı

Odaklı tarayıcıda indirilen içeriğin ilgilenilen konuya ait olup olmadığına karar verilmelidir. Bu yüzden indirilen içerik, etiketini bulmak için sınıflandırılabilir (classification). Odaklı tarayıcıda indirilen içeriğin bir indeksi de oluşturulabilir; böylece bir arama motoru odaklı tarayıcının oluşturduğu indeks üzerinde arama yapar. Yani odaklı tarayıcıda, şimdiye kadar anlatılan normal tarayıcılar gibi bir HTML doküman indeksi oluşturulur, ama bütün dokümanlar aynı konuda olmalıdır.

12.2 Konulu Tarayıcı

Konulu tarayıcı indeks oluşturmaz; web'i arama sırasında tarar. Bir anahtar kelime girip arama düğmesine bastığınızda tarayıcı web'i taramaya başlar. HTML dokümanlarını buldukça içeriklerini tarayıcının konusuyla karşılaştırır:

- Benzerlik yüksekse o dokümandan çıkan URL'leri çıkarmaya ve bu URL'leri izlemeye devam eder.
- Konu ile indirilen doküman arasındaki benzerlik eşikten (threshold) küçükse o sayfadan çıkarılan URL'leri izlemez.

12.3 Uyarlamalı Konulu Tarayıcı

Konulu tarayıcının uyarlamalı (adaptive) bir sürümü de vardır. İndirilen HTML dokümanı ile konu arasındaki benzerlik belirli bir eşiğin üzerindeyse, o dokümandan anahtar kelimeler de çıkarılır ve konuya eklenir.

Hoca daha önceki örneği tekrarladı: "web data mining" ile arama yapan bir konulu tarayıcı bir sayfa indirdi ve sayfanın içeriğini "web data mining" ile karşılaştırdı. Benzerlik yüksekse uyarlamalı tarayıcı o sayfada yüksek sıklıkta geçen diğer anahtar kelimeleri çıkarır. Diyelim ki "information retrieval" da o sayfada yüksek sıklıkta geçiyor. Tarayıcı bunu çıkarıp "web data mining"e ekler. O andan itibaren doküman ile konu arasındaki benzerliği hesaplarken iki anahtar kelime grubunu birlikte kullanır. Yani konuya benzeyen dokümanlardan öğrenir; bu yüzden uyarlamalı konulu tarayıcı denir.

13. Tercihli Tarama Algoritmaları

Hoca farklı tercihli tarama algoritmalarını anlattı. Genişlik öncelikli arama burada yalnızca karşılaştırma için temel (benchmark) olarak yer alır; genişlik ve derinlik öncelikli arama statik tarayıcılardır.

13.1 Best-N-First

URL'ler konuya benzerliklerine göre bir öncelikli kuyruğa (priority queue) konur ve kuyruk URL'lerin konuya ne kadar benzediğine göre sıralanır. Çok iş parçacıklı bir tarayıcımız olduğu için ilk N URL aynı anda taranabilir. Bu yüzden algoritmaya "Best-N-First" denir: En iyi N URL aynı anda taranır.

Ders sonrası not: Altyazıda “best and first” ve “first and URLs” olarak geçen ifadeler “Best-N-First” ve “N URL”dir.

13.2 PageRank

Bu kez URL’ler frontier listesine PageRank değerlerine göre konur. PageRank henüz derste işlenmedi, ama bağlantı tabanlı (link-based) bir popülerlik algoritmasıdır ve her URL’nin popülerliğini verir.

13.3 SharkSearch

SharkSearch, URL’nin metninin benzerliğini ve URL’nin çıkarıldığı üst (parent) sayfanın benzerliğini birleştirir. Hoca bunu tekrarladı:

- URL’nin kendisinden de anahtar kelime çıkarılabildiği için SharkSearch URL’nin içeriğini analiz eder. Hoca buna bağlantı metni (anchor text) dedi.
- Ek olarak URL’nin çıkarıldığı üst sayfanın içeriğine bakar.
- Konu ile bağlantı metni arasındaki ve konu ile üst sayfa içeriği arasındaki benzerliği kontrol eder.
- Buna göre bir benzerlik değeri bulur ve frontier listesini bu benzerlik değerine göre yeniden sıralar.

Ders sonrası not: Hoca bağlantı metnini (anchor text) “URL’nin içeriği” olarak tanımladı. Bağlantı metni aslında bağlantının görünen metnidir, yani HTML’de `<a href="...">` ile `</a>` arasında kalan yazıdır. URL dizgesinin kendi içindeki kelimeler ayrı bir ipucudur. SharkSearch büyük olasılıkla bu ikisini ve üst sayfanın içeriğini birlikte kullanır.

13.4 InfoSpiders

InfoSpiders da bir tercihli tarama algoritmasıdır ve konulu tarayıcıyla ilgilidir. İndeks oluşturmaz. Aldığı HTML dokümanlarından öğrenir: Dokümanlar konuya benziyorsa onlardan yeni anahtar kelimeler çıkarır ve bunları konuya ekler. O andan itibaren yalnızca konu anahtar kelimeleriyle değil, benzer dokümanlardan çıkarılan anahtar kelimelerle de benzerliği kontrol eder. Yani uyarlamalı bir konulu tarayıcıdır.

14. Tercihli Tarayıcıların Değerlendirilmesi

Hoca farklı tercihli tarayıcıların performansının nasıl karşılaştırılacağını gösteren bir grafik gösterdi. Grafikte x ekseni tarama boyutunu (crawl size), y ekseni recall’u gösterir. Hoca recall’u ilgili sayfalarla ilişkilendirerek hatırlattı.

Ders sonrası not: Hoca recall’u “bir konuyla ilgili sayfaların toplam sayısı” diye hatırlattı. Recall bir sayı değil bir orandır: bulunan ilgili sayfa sayısının, konuyla ilgili bütün sayfaların sayısına oranı. Yani $\text{recall} = \dfrac{|\text{bulunan} \cap \text{ilgili}|}{|\text{ilgili}|}$.

Örnekte konu “web data mining” ve dört farklı tarayıcı vardır. Dördü de URL’lerin frontier’a hangi sırayla konacağını farklı belirler; frontier’daki sıra da URL’lerin hangi sırayla alınacağını tanımlar:

- **Rastgele (random):** Frontier listesinden indirmek için herhangi bir URL’yi seçer.
- **Genişlik öncelikli (breadth-first):** İlk giren ilk çıkar; web grafiğini seviye seviye gezer.
- **Geri bağlantı (backlink):** Frontier’daki URL’leri, her URL’nin aldığı bağlantı sayısına, yani geri bağlantı sayısına göre yeniden sıralar.
- **PageRank:** Frontier listesini PageRank değerine göre yeniden sıralar.

Grafiğin yorumu:

- Her şey tarandığında, yani verinin %100’ü tarandığında bütün algoritmaların recall’u %100 olur. Bu açıktır: Her şeyi aldığınızda bütün ilgili dokümanları da almış olursunuz.

- Tarama boyutunun %20'sine bakıldığında, yani alınması gereken toplam dokümanların %20'si alındığında, algoritmalardan biri olan PageRank ilgili sayfaların neredeyse %60'ını almıştır.
- Bu yüzden dört algoritma karşılaştırıldığında PageRank diğer üçünden daha başarılıdır: Taramanın başında, küçük bir tarama boyutunda bile (%20) ilgili sayfaların %60'ını yakalayabilmiştir.

Tercihli tarayıcılar değerlendirilirken tarama boyutu-recall grafikleri kullanılır. Bakılan şey, tarama algoritmasının küçük tarama boyutlarında bile en ilgili sayfaları alıp alamadığıdır.

14.1 Konulu Tarayıcı Arayüzü Örneği

Hoca bir konulu tarayıcının grafik kullanıcı arayüzü örneğini gösterdi. Kullanıcı bir anahtar kelime girer; tarayıcı başlatıldığı anda web'i taramaya başlar ve konuya uyan URL'leri bulur. Bulunan sonuçlara ne kadar benzer olduklarına göre bir puan verir ve sonuçları bu benzerlik puanına göre yeniden sıralar. Hoca konulu tarayıcıların daha fazla ayrıntısına girmedi.

15. Açık Kaynak Tarayıcı Kütüphaneleri

Tarayıcı uygulamalarını gerçekleştirirken yardımcı olacak farklı başvuru kütüphaneleri vardır. Bunlar alttaki (underlying) kütüphanelerdir. Slaytta Perl ve Java sürümleri vardı; slaytta olmayan C# sürümleri de vardır. Hoca Java ile gerçekleştirilmiş Apache Nutch'a bakılmasını önerdi. Bu açık kaynak araçların hepsi eşzamanlı tarayıcı olarak gerçekleştirilmiştir ve derste anlatılan gerçekleştirme sorunlarını da çözer.

> **Ders sonrası not:** Altyazıda "Notch" ve "Natch" diye geçen kütüphane Apache Nutch'tır.

Dersin sonundaki soru-cevapta hoca ayrıca şunu önerdi: Çok karmaşık bir açık kaynak tarayıcı kütüphanesi yerine, derste gösterilen basit yazılım mimarisini gerçekleştiren bir kütüphane seçilmelidir. Örneğin Java'da crawler4j adında bir kütüphane vardır. Daha basit bir gerçekleştirme olursa, gerçekleştirme ayrıntılarında kaybolmadan kendi modülünüzü bu alt kütüphanenin üzerine yerleştirebilirsiniz.

16. Tarayıcı Uygulamalarında Karşılaşılan Pratik Sorunlar (Soru-Cevap)

Dersin sonunda hoca Türkçeye geçti ve tarayıcı geliştirecek gruplar için tarayıcı ile ayrıştırıcının birlikte nasıl çalışacağını anlattı. Böyle bir dikey tarayıcıda iki parça vardır:

- Bir tarayıcı, ya da bir tarayıcı kütüphanesinin üzerine yazılmış bir tarayıcı modülü.
- Bir ayrıştırıcı: E-ticaret sitesinden indirilen HTML verisini ayrıştırıp içinden ilgili içeriği (ürünün fiyatı, adedi, özellikleri ve geldiği sayfanın bağlantısı) çıkarır, böylece bu içerik kendi hibrit uygulamanıza eklenebilir.

Ardından öğrencilerden tarayıcıların pratik kullanımına dair sorular geldi.

Soru: Bir e-ticaret sitesinden belirli bir kategorideki ürünleri çekerken taramaya sitenin ana sayfasından mı başlamalıyız, yoksa telefon gibi belirli kategorilerin sayfalarından başlayabilir miyiz?

Cevap: Sitelerin ana sayfalarında kendi arama arayüzleri vardır. Bu arama arayüzü için bir betik yazıp aradığınız ürünleri aratabilir, oluşan sonuç kümesinin içinden ürünü çekebilirsiniz. Ya da sitenin arayüzündeki gezinme özelliklerini hiç kullanmadan, ana sayfanın URL'sinden başlayıp alt sayfaların tamamını taratabilirsiniz. Bunu elle yapmayacaksınız: Kendiniz bir ürünü aratıp sonuç kümesini çıkarmaya çalışmayın. Ana sayfanın URL'sini verip bütün alt sayfaları taratacak, içlerinden istediğiniz türdeki ürün sayfalarını bulacak ve kendiniz indeksleyeceksiniz. Bazı siteler bir RESTful API arayüzü sunarak arayüzlerinin özelliklerini uzaktan kullanmanıza izin verir; site programlama arayüzüyle filtreleme servisini kullanmanıza izin veriyorsa onu da kullanabilirsiniz.

Soru: Sitelerin sunduğu sayfayı taramak yerine bir REST API kullanabilir miyiz? Siteler çok dinamik olduğu ve JavaScript ile çalıştığı için HTML'i taramak yeterince etkili olmayabilir.

Cevap: Hoca her sitenin böyle bir REST arayüzü sunduğunu düşünmediğini söyledi. Büyük e-ticaret siteleri REST arayüzlerini yalnızca kendi kullanıcı arayüzleri için kullanır. Bu arayüzleri, sitenin gönderdiği HTTP isteklerini izleyerek (monitor) yakalayabilirsiniz, ama bu kez IP sınırlaması getirirler: İstek farklı bir IP'den geliyorsa yanıt vermezler. Site bir REST arayüzü sunsa bile (ki bu düşük bir ihtimaldir), oradan gelen sonuçları kendi sayfanıza entegre etmek için yine ayrıştırmanız gerekir; yani yine bir ayrıştırıcı kullanmak zorundasınız. Hocanın önerisi ve tercihi tarayıcı kullanmaktır: Derste anlatılan gerçekleştirme sorunlarını çözen, belirli bir tohum URL'den başlayıp diğer URL'leri izleyen bir tarayıcı. Hocaya göre içinde JavaScript olsa bile tarayıcı sonuçta HTTP içeriğini, yani tarayıcının (browser) işleyerek bize gösterdiği içeriği yakalar.

Soru: Tarayıcı HTML'i alıp bir sonraki sayfayı `a` etiketinin içinden bulup devam ediyor. Fakat bazı sitelerde ürünler, sayfa aşağı kaydırıldıkça dinamik olarak yükleniyor; ilk gelen HTML'de bütün ürünler olmuyor, yeni etiketler aşağı kaydırdıkça geliyor. Bununla ilgili bir yol gösterebilir misiniz?

Cevap: Hoca bu tür içeriğin gösterilmesinin tarayıcılar için de bir sorun olduğunu söyledi. Nutch'un son sürümünü kullanırsanız bu tür sayfaları da tarayan özellikler getirmiş olmaları gerekir, ama hoca son sürüme bakmadığını belirtti. Önerisi, bu proje için kullanılacak e-ticaret sitelerinin en az JavaScript kullanan, ürünlerini daha çok HTML dokümanlarıyla tanıtan siteler olmasıdır. İçeriğini ağırlıklı olarak JavaScript ile sunan siteler yerine, biraz mühendislik yapıp tarayıcının HTML içeriğini daha iyi yakalayabileceği siteler seçilmelidir.

> **Ek bilgi (derste anlatılmadı):** Aşağı kaydırdıkça içerik yükleme tekniği genellikle "sonsuz kaydırma" (infinite scroll) diye anılır. Sayfa ilk yüklendiğinde gelen HTML yalnızca ilk ürünleri içerir; geri kalanı tarayıcıda çalışan JavaScript'in arka planda yaptığı ek HTTP istekleriyle (genellikle JSON döndüren) getirilir. Sıradan bir tarayıcı JavaScript çalıştırmadığı için bu içeriği görmez; bunun için ya JavaScript çalıştıran başsız (headless) bir web tarayıcısı ya da o ek isteklerin doğrudan yapılması gerekir.

Örnek Sorular

Bu derste kare ya da video olmadığı için sorular yalnızca derste sözlü sorulan sorulardan alınmıştır.

Soru 1 (derste sözlü, 00:14:36): Bir tarayıcı web'i taramayı hangi koşulda durdurur? Durma koşulu ne olabilir?

> **Çözüm (derste verildi):** Durma koşulu tarayıcıya göre değişir. Tarayıcı belirli sayıda sayfa indirdiğinde durabilir; tohum URL'lerden başlayarak tarama derinliği belirli bir seviyeye (örneğin 3. seviye) ulaşınca durabilir; ya da frontier listesinde hiç URL kalmayınca durabilir. Perl örneğinde koşul, indirilen sayfa sayısının yapılandırılabilir bir üst sınırdan küçük olması ve frontier'ın boş olmamasıdır.

Soru 2 (derste sözlü, 00:18:05): Neden tarayıcılara ihtiyaç duyarız?

> **Çözüm (derste verildi):** Arama motorlarında gerçek zamanlı arama ters indeks üzerinde yapılır; indeksin oluşturulabilmesi için sayfaların önce merkezi bir depoya indirilmesi gerekir. Bunun dışında dikey/özelleşmiş tarayıcılar (haber, alışveriş, makale, yemek tarifi, yorum siteleri), iş zekâsı için rakiplerin izlenmesi, ilgilenilen sitelerin değişiklikler için izlenmesi ve kötü amaçlı olarak e-posta adresi toplama (spam, oltalama) da tarayıcı kullanım alanlarıdır.

Soru 3 (derste sözlü, 01:08:55): Bir HTML sayfasından çıkarılan mutlak URL ile göreli URL arasındaki fark nedir?

> **Çözüm (derste verildi):** Mutlak URL `http` ile başlayan tam adrestir ve tek başına bir sayfayı gösterir. Göreli URL, sayfa yazarının aynı alan adı içindeki bağlantılar için yazdığı kısa biçimdir. Göreli URL, üst sayfanın temel URL'siyle birleştirilerek mutlak URL'ye çevrilmelidir; frontier'a yalnızca mutlak (ve kanonik) URL'ler konur.

Soru 4 (derste sözlü, 01:37:25): Bir web sayfasının tazeliğini nasıl tanımlarız?

> **Çözüm (derste verildi):** HTTP yanıt başlığındaki `Last-Modified` ve `Expires` alanlarıyla. `Last-Modified` sayfanın en son ne zaman güncellendiğini, dolayısıyla indirilen

kopyanın güncel olup olmadığını gösterir. `Expires` içeriğin geçersiz olacağı tarihi verir; sayfa o tarihte yeniden taranmalıdır.

Soru 5 (derste sözlü, 02:01:44): Bir e-ticaret sitesinden belirli bir kategorideki ürünler toplanırken tarama ana sayfadan mı yoksa kategori sayfasından mı başlatılmalıdır?

> **Çözüm (derste verildi):** Sitenin arama arayüzünü elle kullanarak sonuç kümesi çıkarılmamalıdır. Ana sayfanın URL'si tohum URL olarak verilip bütün alt sayfalar taranır, istenen türdeki ürün sayfaları bulunup kendi indeksinizde toplanır. Site bir programlama arayüzü (REST API) ile filtreleme servisine izin veriyorsa o da kullanılabilir.

Soru 6 (derste sözlü, 02:07:08): Ürünlerin sayfa aşağı kaydırıldıkça dinamik olarak yüklendiği sitelerde tarayıcı ne yapmalıdır?

> **Çözüm (derste verildi):** Bu tür sayfalar tarayıcılar için bir sorundur. Nutch'un yeni sürümlerinde bu sayfaları tarayan özellikler olabilir. Hocanın önerisi, JavaScript'i az kullanan ve ürünlerini HTML dokümanlarıyla sunan sitelerin seçilmesidir.

Dersin Toparlaması

Bu derste bir web tarayıcısının ne olduğu, neden gerektiği ve nasıl gerçekleştirileceği baştan sona ele alındı. Tarayıcı, aramadan önce arka planda çalışarak tohum URL'lerden başlar, sayfaları indirir, içlerinden yeni URL'ler çıkarıp frontier listesine ekler ve sayfaları indeksin oluşturulacağı depoya yazar. Temel sıralı tarayıcının modülleri (frontier'ı başlatma, dequeue, fetch, parse, store) ve frontier'ın kuyruk ya da yığın olarak kullanılmasına göre ortaya çıkan genişlik ve derinlik öncelikli gezinme gösterildi. Basit görünen bu döngünün her adımında çözülmesi gereken sorunlar vardır: tekrar indirmeyi önlemek, frontier'ı önceliklendirmek, fetcher'a zaman aşımı, içerik türü kontrolü ve boyut sınırı koymak, bozuk HTML'i DOM ağacıyla ayrıştırmak, URL'leri mutlak ve kanonik biçime getirmek, örümcek tuzaklarından kaçınmak ve sayfaları verimli saklamak.

Ardından çok iş parçacıklı eşzamanlı tarayıcıda frontier'ın paylaşılan veri yapısı olduğu ve senkronizasyonun frontier yöneticisinde yapıldığı, evrensel tarayıcılarda kapsam ile tazelik arasındaki denge ve DNS önbelleği, asenkron soketler gibi hızlandırma çözümleri anlatıldı. Tercihli tarayıcılarda frontier konuya benzerliğe göre sıralanır; Best-N-First, PageRank, Shark-Search ve InfoSpiders algoritmaları tanıtıldı ve tarayıcıların tarama boyutu-recall grafiğiyle değerlendirildiği gösterildi. Bu grafikte PageRank'e göre sıralama, taramanın %20'sinde ilgili sayfaların yaklaşık %60'ını bularak en başarılı strateji oldu. Gelecek hafta bağlantı analizi (link analysis) işlenecek; hoca bunun önemli bir konu olduğunu ve sınavda çıkacağını belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.